

Технологично училище „Електронни системи“ към ТУ-София

Тема: Мрежови сървър/клиент на TCP с нишки
(pthreads)



Ученик: Виктория Ганчева 11г №7

1. Въведение

Проектът представлява мрежова игра „Състезателна бесеница“, реализирана чрез TCP клиент и сървър. Играта се стартира с два отделни клиента и един сървър.

Всеки играч избира дума, която другият играч трябва да отгатне. Двамата играчи играят едновременно. Когато и двамата познаят думите си, сървърът сравнява броя на грешните предположения. Играчът с по-малко грешки печели. Ако броят на грешките е еднакъв, играта завършва наравно.

Решението се състои от два изпълними файла:

hangman-server

hangman-client

2. Обща идея на решението

Програмата работи на принципа клиент-сървър. Сървърът чака да се свържат двама играчи. Всеки клиент изпраща дума, която опонентът му ще трябва да отгатва.

След като сървърът получи валидни думи и от двамата клиенти, той ги разменя. Това означава, че първият клиент отгатва думата на втория, а вторият клиент отгатва думата на първия.

Сървърът не изпраща цялата дума към клиента, а само нейното текущо състояние. Неразкритите букви се показват с долна черта `_`. Така клиентът не може да види думата предварително.

3. Сървър

Сървърът се стартира по следния начин:

```
./hangman-server <port>
```

Например:

```
./hangman-server 9000
```

След стартиране сървърът създава TCP socket и започва да слуша на адрес `0.0.0.0` и подадения порт. Когато е готов, извежда:

Listening on 9000...

Сървърът приема точно двама клиента. След като двамата се свържат успешно, повече връзки не се приемат.

При свързване всеки клиент изпраща дума. Сървърът проверява дали думата е валидна чрез функцията:

```
secret_word_init_from_c_string()
```

Ако думата е невалидна, например съдържа цифра или друг символ, сървърът връща грешка и затваря връзката:

ERR invalid word

Когато и двете думи са валидни, сървърът изпраща:

OK

След това сървърът създава по една нишка за всеки клиент. Всяка нишка обслужва играта на един играч. Това позволява двамата играчи да играят едновременно.

4. Клиент

Клиентът се стартира по следния начин:

```
./hangman-client <host> <port> <opponent-word>
```

Пример:

```
./hangman-client 127.0.0.1 9000 apple
```

Третият аргумент е думата, която опонентът ще трябва да отгатне.

След като се свърже към сървъра, клиентът изпраща думата си и чака отговор. Ако получи OK, започва играта.

По време на играта клиентът показва текущото състояние на думата и грешните букви. Изходът изглежда така:

Word: h_ll_

Incorrect guesses: a, i, c

След това играчът въвежда една буква. Клиентът я изпраща към сървъра и получава новото състояние на играта.

Цикълът продължава, докато думата бъде позната изцяло.

5. Комуникация между клиента и сървъра

Комуникацията е текстова. Всяко съобщение завършва с нов ред `\n`.

5.1. Начало на връзката

Първо клиентът изпраща думата си:

apple

Ако думата е валидна, сървърът отговаря:

OK

Ако има грешка:

ERR invalid word

5.2. По време на игра

Сървърът изпраща текущото състояние на думата:

WORD h_ll_

След това изпраща грешните букви:

WRONG a, i, c

Клиентът въвежда буква, например:

e

Сървърът обработва буквата и връща новото състояние.

Когато думата е напълно позната, сървърът изпраща:

SOLVED

5.3. Краен резултат

След като и двамата играчи приключат, сървърът изпраща резултат:

FINAL WIN

или:

FINAL LOSE

или:

FINAL TIE

След това изпраща грешните букви на играча и на опонента:

YOUR x, z

OPP a, b, c

Клиентът ги показва така:

YOU WIN! :)

Your incorrect guesses: x, z

Opponent's incorrect guesses: a, b, c

6. Примерна комуникация

Пример с двама играчи:

- Играч А задава дума moon;
- Играч В задава дума sun;
- Играч А трябва да отгатва sun;
- Играч В трябва да отгатва moon.

Примерна последователност:

1. Играч А изпраща към сървъра думата **moon**.
2. Играч В изпраща към сървъра думата **sun**.
3. Сървърът проверява дали и двете думи са валидни.
4. Ако думите са валидни, сървърът връща **OK** и на двамата играчи.
5. Сървърът разменя думите между играчите.
6. Играч А започва да отгатва думата **sun** и вижда:

WORD s__

WRONG

7. Играч В започва да отгатва думата **moon** и вижда:

WORD m__

WRONG

8. Играч А въвежда буквите **t, u, n**.
9. Играч В въвежда буквите **o, o, n**.
10. След всяка въведена буква сървърът връща обновеното състояние на думата.
11. Когато Играч А познае думата, той вижда:

WORD sun

SOLVED

12. Когато Играч В познае думата, той вижда:

WORD moon

SOLVED

13. След като и двамата играчи приключат, сървърът сравнява броя на грешните предположения.
14. В този пример няма грешни букви и при двамата играчи, затова резултатът е равенство:

FINAL TIE

Клиентът показва крайния резултат така:

Tie :/

Your incorrect guesses:

Opponent's incorrect guesses:

7. Синхронизация на нишките

Сървърът използва две нишки, по една за всеки клиент. За да може финалният резултат да се изпрати чак след края и на двете игри, се използват `mutex` и условна променлива.

В сървъра има обща структура, която пази информация за двамата играчи. В нея има:

```
player_t players[2];
int solved[2];
pthread_mutex_t mutex;
pthread_cond_t finished_cond;
```

Масивът `solved[2]` показва дали даден играч вече е приключил. Например `solved[0] = 1` означава, че първият играч е готов.

Когато дадена нишка приключи, тя:

1. заключва `mutex`-а;
2. отбелязва, че нейният играч е готов;
3. събужда другата нишка чрез `pthread_cond_broadcast()`;
4. изчаква, докато и другият играч приключи;
5. отключва `mutex`-а;
6. изпраща крайния резултат към клиента.

Така резултатът се показва едва когато и двамата играчи са приключили. Не се използва постоянно проверяване в цикъл, тоест няма активно чакане.

8. Компиляция и стартиране

Компиляцията се прави с:

```
make all
```

След това в основната папка се създават:

```
hangman-server
```

```
hangman-client
```

Стартиране на сървъра:

```
./hangman-server 9000
```

Стартиране на първи клиент:

```
./hangman-client 127.0.0.1 9000 moon
```

Стартиране на втори клиент:

```
./hangman-client 127.0.0.1 9000 sun
```

Важно е думите да са с латински букви от а до z, защото предоставената библиотека `game.c/game.h` работи с такива букви.

В основната директория има `Makefile`, като първият `target` е `all`. Той компилира проекта и създава двата изпълними файла `hangman-server` и `hangman-client`.

9. Използвани функции и библиотеки

Функция / библиотека

За какво се използва

`socket()`

Създаване на `socket`

`bind()`

Свързване на `socket` към адрес и порт

`listen()`

Сървърът започва да слуша за клиенти

`accept()`

Приемане на клиентска връзка

<code>connect()</code>	Свързване на клиента към сървъра
<code>send()</code>	Изпращане на данни
<code>recv()</code>	Получаване на данни
<code>close()</code>	Затваряне на връзка
<code>pthread_create()</code>	Създаване на нишка
<code>pthread_join()</code>	Изчакване на нишка
<code>pthread_mutex_lock()</code>	Заклучване на mutex
<code>pthread_mutex_unlock()</code>	Отключване на mutex
<code>pthread_cond_wait()</code>	Изчакване чрез условна променлива
<code>pthread_cond_broadcast()</code>	Събуждане на чакащи нишки
<code>setsockopt()</code>	Позволява повторно използване на порт
<code>signal(SIGPIPE, SIG_IGN)</code>	Предпазва от прекъсване при затворена връзка
<code>setvbuf()</code>	Използва се клиентът да печата веднага изхода си
<code>secret_word_init_from_c_string()</code>	Проверява и създава дума
<code>secret_word_guess()</code>	Обработва позната буква
<code>secret_word_letter_at()</code>	Проверява дали буква е разкрита
<code>secret_word_is_solved()</code>	Проверява дали думата е позната
<code>secret_word_incorrect_guess_count()</code>	Връща броя грешни предположения

10. Заключение

Проектът реализира TCP клиент и сървър за игра „Състезателна бесеница“. Сървърът приема двама играчи, разменя думите им и управлява играта с две нишки. Клиентът показва текущото състояние на думата, приема буква от потребителя и накрая извежда резултата.

Решението използва предоставената библиотека `game.c/game.h` за игровата логика, а мрежовата комуникация е реализирана чрез socket-и. За синхронизация между двете нишки се използват mutex и условна променлива.